Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification – Self-Service Coffee Kiosk System

Architecture Selection

We chose **Layered Architecture** for the **Self-Service Coffee Kiosk** System.

Architectural Choice

The kiosk is built as a three-layer system: a **Presentation Layer** (User Interface Component), a **Business Layer** (Order Manager, Payment Service and Receipt Printer Components) and a **Data Layer** (Menu Database Component). Components talk only through four interfaces: IOrderService, IPaymentService, IReceiptPrinter and IMenuData.

Two Reasons for the Choice

- **Requirements map directly onto layers.** The kiosk is one device with a small, fixed scope (3 coffees, 2 sizes, card payment only). The touch screen, the order logic with payment and printing, and the menu/pricing storage each fit one layer, so every component has a single responsibility and dependencies flow in one direction only (UI → Order Manager → Payment / Printer / Database). Microservices would add network calls and deployment overhead that a single kiosk does not need, and a client-server design would make the kiosk depend on a central server, creating a single point of failure.
- **Easy to maintain and replace parts.** Because components are black boxes that interact only through interfaces, the receipt printer model or payment provider can be swapped, or the menu moved from a local database to a cloud one, without changing the UI or Order Manager. Adding a new coffee type or size is only a data change in the Data Layer.

Security Advantage

The Order Manager is the single controlled entry point between the layers: the touch-screen UI never accesses the Payment Service or the database directly. The Order Manager validates every request (valid drink, size and price) before acting on it, which stops tampering with prices or orders from the UI. Card details are confined to the Payment Service Component, which communicates with the payment processor over an encrypted (TLS) channel; the Order Manager only receives an approved/declined result, and receipts show only masked card digits. This isolation limits where sensitive data exists and makes PCI-DSS compliance easier to achieve.

Performance Benefit

All layers run on the same kiosk, so calls between components are in-process method calls with no network hops, which keeps the touch screen responsive. The menu and pricing data are small and can be loaded once by the Order Manager and cached, so selecting a drink needs no repeated database reads. Only payment authorisation depends on the network, and receipt printing is handled by its own component, so the UI can show the order confirmation without waiting on the printer driver.

Trade-off Acknowledged

Layering adds a small amount of indirection and makes it harder to scale individual components independently. For a single-device kiosk with a low and predictable load, this cost is negligible compared to the simplicity and security gained.